

条款21 优先考虑使用 `std::make_unique` 和 `std::make_shared`

① 减少代码重复

`auto upw1(std::make_unique<Widget>());` 写1次

`std::unique_ptr<Widget> upw2(new Widget);` 写2次

`shared` 同理

② 使用 `make_xxx` 更安全

声明: `void processWidget(std::shared_ptr<Widget> spw,  
int priority);`

调用: `processWidget(std::shared_ptr<Widget>(new Widget),  
computePriority());`  
③ ① ②

②与③不知谁先进行,但是如果②在③前进行且②抛出异常 ①内存泄露 出大问题

改成: `processWidget(std::make_shared<Widget>(),  
computePriority());` 解决问题  
② 将②与③步骤聚合并

绿色代码存在异常是因为①与③的执行被②异常打断

所以只要不让被打断,即使不使用 `make_shared`,也能解决问题

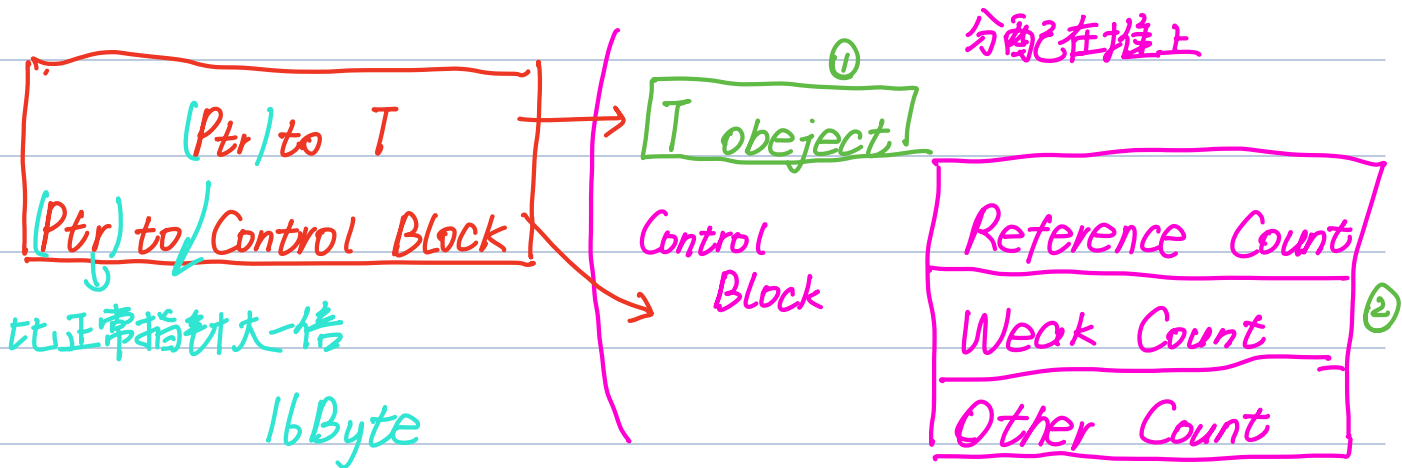
`std::shared_ptr<Widget> spw(new Widget);`

`processWidget(spw, computePriority());`

此处可能存在 copy

使用 `std::move(spw)`

③ `std::make_shared` 与 `new` 相比效率上有提升



①与②都会分配到堆上,如果使用`new`,会向堆申请两次内存  
而使用`make_shared`会一次性将所需的内存申请下来

④ `make` 的局限性 (a, b 是通病 c, d `make_shared` 独有)

a. 使用自定义删除器时只能使用 `new`

b. 无法通过 {} 初始化指向的对象 (原因: {} 无法完美转发, 条款 30)

`auto spv = std::make_share<vector<int>>({1, 2, 3});` 报错

c. 如果类重载了 `operator new/delete`, 使用 `make_share` 不会执行重载函数, 此时只能使用 `share_ptr` / `std::allocated_shared`

d. 使用 `make_shared` T object 与 Control Block 一起申请, 一起释放

所以 `weak_ptr` 存在时, 对象的销毁与内存释放的间隔可能很长

```
auto sptr = std::make_shared<Widget>();
```

```
std::weak_ptr wptr(sptr);
```

```
sptr = nullptr;
```

时间轴:  $t0: \text{make\_shared} \rightarrow T \text{ object} + \text{Control Block}$

强计数=1 弱计数=0

$t1: \text{构造 } wptr \rightarrow \text{强计数}=1 \text{ 弱计数}=1$

$t2: sptr = nullptr \rightarrow \text{对象被销毁 } T \text{ object}$

间隔 ↓

强计数=0 弱计数=1

$t3: wptr \text{ 离开作用域} \rightarrow \text{内存被释放 } \text{Control Block}$

强、弱计数=0 内存完全回收